

OneTrans: 以单一 Transformer 统一工业推荐中的特征交互与序列建模

Zhaoqi Zhang*

Nanyang Technological

University

ByteDance

Singapore, Singapore

zhaoqi.zhang@bytedance.com

Haolei Pei*

ByteDance

Singapore, Singapore

haolei.pei@bytedance.com

Jun Guo*

ByteDance

Singapore, Singapore

jun.guo@bytedance.com

Tianyu Wang

ByteDance

Singapore, Singapore

tianyu.wang01@bytedance.com

Yufei Feng

ByteDance

Hangzhou, China

fengyihui@bytedance.com

Hui Sun

ByteDance

Hangzhou, China

sunhui.sunh@bytedance.com

Shaowei Liu[†]

ByteDance

Singapore, Singapore

liushaowei.nphard@bytedance.com

Aixin Sun[†]

Nanyang Technological

University

Singapore, Singapore

axsun@ntu.edu.sg

摘要

在推荐系统中,扩展特征交互模块(例如 Wukong、RankMixer)或用户行为序列模块(例如 LONGER)已取得显著成功。然而,这些工作通常沿着彼此独立的路线推进,这不仅阻碍了双向信息交换,也无法实现统一优化与扩展。本文提出 ONETRANS,一种同时进行用户行为

序列建模和特征交互的统一 Transformer 骨干网络。

ONETRANS 采用统一的分词器,将序列属性与非序列属性转换为单一 token 序列。堆叠的 ONETRANS 块在相似的序列 token 之间共享参数,同时为非序列 token 分配 token 特定参数。通过因果注意力与跨请求 KV 缓存,ONETRANS 能够预计算并缓存中间表示,从而显著降低训练和推理阶段的计算成本。在工业规模数据集上的实验结果表明,ONETRANS 能随参数量增加而高效扩展,持续优于强基线,并在在线 A/B 测试中使人均 GMV 提升 5.68%。

*这些作者贡献相同。

[†]通讯作者。



This work is licensed under a Creative Commons Attribution 4.0 International License.

会议简称 'XX, 纽约州伍德斯托克

© 2025 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-XXXX-X/18/06

CCS Concepts

• 信息系统 → 信息检索; 推荐系统;

Keywords

推荐系统, 排序模型, 扩展定律

ACM Reference Format:

Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2025. OneTrans: 以单一 Transformer 统一工业推荐中的特征交互与序列建模. In *Proceedings of* 请务必填写版权确认邮件中的正确会议名称 (会议简称'XX'). ACM, New York, NY, USA, 13 pages.

1 引言

推荐系统 (RecSys) 在电子商务 [9, 35]、流媒体 [2, 20, 28] 和社交网络 [31, 32] 等各种信息服务中发挥着基础性作用。工业推荐系统通常采用级联排序架构 [6, 16, 22]。首先, 召回阶段从十亿规模的语料库中选出数百个候选项 [13, 36]。随后, 排序阶段为每个候选项打分, 并返回前 k 个物品 [11, 27, 28, 32, 37]。深度学习推荐模型 (DLRM) [19] 已被广泛用于工业推荐系统的排序阶段。

本文遵循 DLRM 风格的排序范式, 聚焦于排序阶段。对于排序, 主流方法围绕两个独立模块进行迭代: (a) 序列建模, 使用局部注意力或 Transformer 编码器将用户多行为序列编码为候选感知表示 [1, 14, 25, 35]; 以及 (b) 特征交互, 通过因子分解、显式交叉网络或特征组上的注意力, 学习非序列特征 (例如用户画像、物品画像和上下文) 之间的高阶交叉关系 [11, 12, 27, 37]。如图 1(a) 所示, 这些方法通常先将用户行为编码为一个压缩的序列表示, 再将其与非序列特征拼接, 并应用特征交互模块来学习高阶交互; 我们将这种设计称为先编码后交互流水线。

大语言模型 (LLM) 的成功表明, 扩大模型规模 (例如参数规模、训练数据量) 能够带来可预测的性能提升 [15], 这也启发了推荐系统领域的类似研究 [1, 32, 37]。在特征交互方面, Wukong [32] 堆叠带线性压缩的因子分解机模块, 以捕获高阶特征交互并建立缩放定律; RankMixer [37] 则通过硬件友好的 token 混合以及 token 特定的前馈网络 (FFN), 获得良好

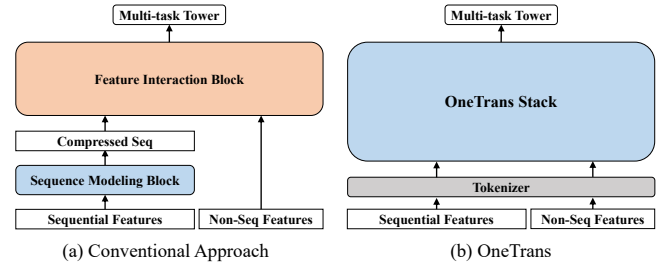


图 1: 架构对比。(a) 传统的先编码、后交互流程先编码序列特征并合并非序列特征, 再通过后置的特征交互块进行处理。(b) OneTrans 在单个 OneTrans (Transformer 风格) 堆栈内对序列特征和非序列特征进行联合建模。

的规模扩展能力。在序列建模方面, LONGER[1] 将因果 Transformer 应用于较长的用户历史, 并表明增加深度和宽度可带来单调的性能提升。尽管在实践中有效, 但将序列建模与特征交互拆分为独立模块会带来两个主要局限。首先, 先编码后交互流水线限制了双向信息流, 从而限制了静态/上下文特征对序列表示的塑造能力 [30]。其次, 模块分离会造成执行碎片化并增加延迟, 而单一 Transformer 风格的骨干网络能够复用 LLM 优化, 例如 KV 缓存、内存高效注意力和混合精度, 从而实现更有效的规模扩展 [11]。

本文提出 **OneTrans**, 这是一种创新的架构范式, 采用统一的 Transformer 骨干网络联合执行用户行为序列建模与特征交互。如图 1(b) 所示, ONETRANS 在统一骨干网络内部实现双向信息交换。它采用统一分词器, 将序列特征 (多种行为序列) 和非序列特征 (静态用户/物品特征及上下文特征) 转换为单一 token 序列, 随后由堆叠的 ONETRANS 模块组成的金字塔进行处理; 该模块是一种面向工业推荐系统定制的 Transformer 变体。为适应推荐系统中多样的 token 来源 (不同于 LLM 中纯文本 token), 每个 ONE-TRANS 模块采用与 HiFormer [11] 类似的混合参数化方式。具体而言, 所有序列 token (来自序列特征) 共享同一组 Q/K/V 和 FFN 权重, 而每个非序列 token (来自非序列特征) 都使用 token 特定参数, 以保留其独特语义。

不同于传统的先编码后交互框架, ONETRANS 通过统一的因果 Transformer 骨干网络, 消除了序列特征与非序列特征之间的架构屏障。这种形式使推荐系统的规模扩展与 LLM 实践保持一致: 可以通过调整骨干网络的深度和宽度来扩展整个模型, 同时无缝继承成熟的 LLM 优化, 例如 FlashAttention [7] 和混合精度训练 [17]。特别地, 跨候选和跨请求 KV 缓存 [1] 可将含有 C 个候选项的会话的时间复杂度从 $O(C)$ 降至 $O(1)$, 使大规模部署 ONETRANS 成为可能。

总而言之, 我们的主要贡献有以下四点: **(1) 统一框架。**我们提出 ONETRANS, 一个用于排序的单一 Transformer 骨干网络; 它配备统一分词器, 将序列和非序列特征编码为一个 token 序列, 并采用统一 Transformer 模块联合执行序列建模与特征交互。**(2) 面向推荐系统的定制。**为弥合 LLM 与推荐系统任务之间的差距, ONETRANS 引入混合参数化, 为多样的非序列 token 分配 token 特定参数, 同时为所有序列 token 共享参数。**(3) 高效训练与服务。**我们通过逐步剪枝序列 token 的金字塔策略, 以及在候选项之间复用用户侧计算的跨请求 KV 缓存来提高效率。此外, 我们采用 FlashAttention、混合精度训练和半精度推理等 LLM 优化, 进一步减少内存和计算开销。**(4) 规模扩展与部署。**随着模型规模增大, ONETRANS 展现出近似对数线性的性能提升, 为真实生产数据中的缩放定律提供了证据。在线部署后, 它在保持生产级延迟的同时, 使业务 KPI 获得了具有统计显著性的提升。

2 相关工作

早期推荐系统 (RecSys) 如 DIN [35] 及其具备会话感知能力的变体 (DSIN) [9] 使用局部注意力学习以候选项为条件的用户历史摘要, 但会将每个候选项的行为压缩为定长向量, 限制了长程依赖建模 [34]。SAS-Rec [14]、BERT4Rec [25] 和 BST [4] 等自注意力方法允许每个位置关注完整历史, 从而消除了这一瓶颈, 并通过双向掩码提高样本效率。近年来, 随着推荐系统中的扩展定律 [15] 日益受到探索, LONGER [1] 通

过面向超长行为历史的高效注意力与服务友好型设计, 将序列建模推向工业规模。然而, 在主流流程中, 这些序列编码器通常仍与特征交互堆栈相互分离, 因而形成后期融合, 而非与静态上下文特征进行联合优化 [30]。

在特征交互方面, 早期推荐系统依赖人工设计的交叉特征或自动乘法交互层。Wide&Deep [5]、FM/DeepFM [3, 12] 和 DCN/DCNv2 [26, 27] 等经典模型能够高效实现低阶或有界阶数的交互。然而, 正如近期扩展研究所观察到的 [32], 一旦模型堆叠了足够多的交叉层, 继续增加层数便不再带来收益: 模型质量趋于饱和, 而非持续提升。为克服预设交叉形式的僵化, 基于注意力的方法会自动学习高阶交互。AutoInt [24] 学习任意阶关系, 而 HiFormer [11] 引入组特定投影, 以更好地捕获异构、非对称交互。随着扩展越来越多地应用于特征交互模块, Wukong [32] 等大规模系统通过堆叠带线性压缩的 FM 风格交互块, 展现出可预测的收益, 而 RankMixer [37] 则在严格延迟预算下, 借助并行 token 混合和稀疏 MoE 实现了良好的扩展能力。然而, 这些交互模块通常遵循交互范式, 将交互推迟到一个独立阶段, 从而阻碍了与用户序列建模的统一优化 [30]。

迄今为止, 推荐系统的进展主要沿两条独立路线推进: 序列建模与特征交互。InterFormer [30] 尝试通过基于摘要的双向交叉架构弥合这一差距, 使两个组件能够相互交换信号。然而, 它仍将二者作为独立模块, 且交叉架构同时带来了架构复杂性和执行碎片化。由于缺少用于联合建模与优化的统一骨干网络, 系统仍难以作为一个整体进行扩展。

近期关于生成式推荐器 (GR) 的工作将推荐表述为序列转导, 并提出了 HSTU [31] 等高效长上下文骨干网络。这一路线与依赖丰富非序列 (NS) 特征的 DLRM 形成互补。

3 方法

在详细介绍我们的方法之前, 先简要说明任务设定。在级联式工业推荐系统中, 每当召回阶段为用户 u 返回一个候选集合 (通常包含数百个候选物品) 时, 排

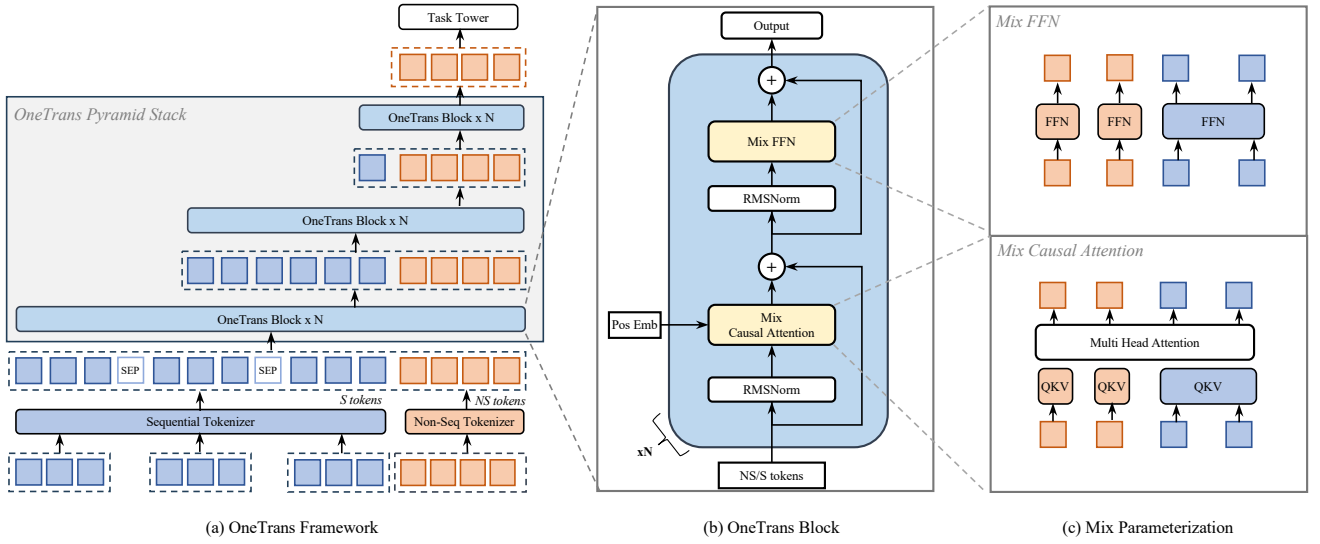


图 2: 系统架构。(a) OneTrans 概览。序列 (S, 蓝色) 与非序列 (NS, 橙色) 特征分别进行 token 化。在用户行为序列之间插入 [SEP] 后, 统一 token 序列被送入堆叠的 OneTrans 金字塔模块, 逐步缩短 token 长度, 直至其与 NS token 的数量一致。(b) OneTrans 模块: 一个采用 RMSNorm、混合因果注意力和混合 FFN 的因果 Pre-Norm Transformer 模块。(c) “混合”即混合参数化: S token 共享一组 QKV/FFN 权重, 而每个 NS token 均使用其各自的 token 特定 QKV/FFN。

序模型便为每个候选物品 i 预测一个分数:

$$\hat{y}_{u,i} = f(i | \mathcal{NS}, \mathcal{S}; \Theta) \quad (1)$$

其中, $\mathcal{NS}$ 是由用户、候选物品和上下文派生的非序列特征集合; $\mathcal{S}$ 是用户的历史行为序列集合; Θ 为可训练参数。常见的任务预测包括点击率 (CTR) 和点击后转化率 (CVR)。

$$\begin{aligned} \text{CTR}_{u,i} &= P(\text{click} = 1 | \mathcal{NS}, \mathcal{S}; \Theta), \\ \text{CVR}_{u,i} &= P(\text{conv} = 1 | \text{click} = 1, \mathcal{NS}, \mathcal{S}; \Theta). \end{aligned} \quad (2)$$

3.1 OneTrans 框架概述

如图 2(a) 所示, ONETRANS 采用一个统一分词器, 将序列特征 $\mathcal{S}$ 映射为 S-token, 并将非序列特征 $\mathcal{NS}$ 映射为 NS-token。随后, 一个金字塔堆叠 Transformer 在单一计算图中联合处理统一的 token 序列。我们将初始 token 序列记为

$$\mathbf{X}^{(0)} = [\text{S-tokens}; \text{NS-tokens}] \in \mathbb{R}^{(L_S + L_{NS}) \times d}. \quad (3)$$

该 token 序列由 L_S 个 S-token 与 L_{NS} 个 NS-token 拼接而成, 所有 token 的维度均为 d 。需要注意的是, S-token 中插入了可学习的 [SEP] token, 用于标定不同类型用户行为序列之间的边界。如图 2(b) 所示, 每个 ONETRANS 块通过以下过程逐步细化 token 状态:

$$\mathbf{Z}^{(n)} = \text{MixedMHA}(\text{Norm}(\mathbf{X}^{(n-1)})) + \mathbf{X}^{(n-1)}, \quad (4)$$

$$\mathbf{X}^{(n)} = \text{MixedFFN}(\text{Norm}(\mathbf{Z}^{(n)})) + \mathbf{Z}^{(n)}. \quad (5)$$

其中, MixedMHA (混合多头注意力) 和 MixedFFN (混合前馈网络) 采用混合参数化策略 (见图 2(c)): 序列 token 共享权重, 而非序列 token 在注意力层和前馈层中均使用各自独立的参数。

统一的因果掩码施加自回归约束, 使每个位置只能关注其之前的 token。具体而言, NS-token 可以关注 S-token 的全部历史, 从而实现全面的跨 token 交互。通过堆叠此类模块, 并对 S-token 进行金字塔式尾部截断, 模型逐步将紧凑的高阶信息提炼到 NS-token 中。最终的 token 状态随后传入任务专用预测头进行预测。

通过将非序列特征和序列特征统一为一个 token 序列, 并使用因果 Transformer 对其建模, ONETRANS 摒弃了传统的先编码、后交互流程。这种统一设计自然支持: (i) 每个行为序列内部的序列内交互; (ii) 多个序列之间的跨序列交互; (iii) 物品、用户与上下文特征之间的多源特征交互; 以及 (iv) 序列-特征交互, 且所有交互均在单个 Transformer 堆栈内完成。

统一表述使我们能够无缝继承成熟的 LLM 工程优化, 包括 KV 缓存和内存高效注意力, 从而显著降低推理延迟。我们认为, 这种统一表述非常适合以单一且可扩展的架构应对多序列和跨域推荐挑战。下面将详细介绍其设计。

3.2 特征与分词

为构建初始 token 序列 $\mathbf{X}^{(0)}$, ONETRANS 首先应用特征预处理流水线, 将所有原始特征输入映射为嵌入向量。随后, 这些嵌入被划分为: (i) 多行为序列子集; 以及 (ii) 表示用户、物品或上下文特征的非序列子集。两个子集分别应用不同的分词器。

3.2.1 非序列分词. 非序列特征 $\mathcal{NS}$ 同时包括数值输入 (例如价格、CTR) 和类别输入 (例如用户 ID、物品类别)。所有特征均先进行分桶或独热编码, 再进行嵌入。由于工业系统通常包含数百个重要性各异特征, 因此有两种方案可控制非序列 token 的数量 (记为 L_{NS}):

分组式分词器 (与 RankMixer [37] 一致)。将特征人工划分为若干语义组 $\{\mathbf{g}_1, \dots, \mathbf{g}_{L_{NS}}\}$ 。每组特征拼接后送入该组专用的 MLP:

$$\text{NS-tokens} = [\text{MLP}_1(\text{concat}(\mathbf{g}_1)), \dots, \text{MLP}_{L_{NS}}(\text{concat}(\mathbf{g}_{L_{NS}}))] \quad (6)$$

自动拆分分词器. 另一种方案是将所有特征拼接起来, 通过单个 MLP 进行一次投影, 然后再拆分:

$$\text{NS-tokens} = \text{split}(\text{MLP}(\text{concat}(\mathcal{NS})), L_{NS}). \quad (7)$$

与分组式方案相比, 自动拆分分词器使用单次稠密投影, 从而减少了核函数启动开销。我们将在实验中评估这两种选择。

最终, 非序列分词得到 L_{NS} 个非序列 token, 每个 token 的维度为 d 。

3.2.2 序列分词. ONETRANS 接收如下形式的多行为序列:

$$\mathcal{S} = \{\mathbf{S}_1, \dots, \mathbf{S}_n\}, \quad \mathbf{S}_i = [\mathbf{e}_{i1}, \dots, \mathbf{e}_{iL_i}]. \quad (8)$$

每个序列 $\mathbf{S}_i$ 包含 L_i 个事件嵌入 $\mathbf{e}$, 每个事件嵌入由物品 ID 与相应的辅助信息 (如物品类别和价格) 拼接而成。

多行为序列的原始维度可能不同。因此, 对于每个序列 $\mathbf{S}_i$, 我们使用一个共享投影 MLP_i , 将其所有事件 $\mathbf{e}_{ij}$ 转换到共同维度 d :

$$\tilde{\mathbf{S}}_i = [\text{MLP}_i(\mathbf{e}_{i1}), \dots, \text{MLP}_i(\mathbf{e}_{iL_i})] \in \mathbb{R}^{L_i \times d}. \quad (9)$$

对齐后的序列 $\tilde{\mathbf{S}}_i$ 按以下两种规则之一合并为单个 token 序列: 1) 时间戳感知: 按时间交错排列所有事件, 并加入序列类型指示符; 2) 时间戳无关: 按事件影响力拼接序列, 例如购买 $\rightarrow$ 加购 $\rightarrow$ 点击, 并在序列之间插入可学习的 [SEP] token。在后一种规则中, 体现更强用户意图的行为被置于序列前部。消融结果表明, 当时间戳可用时, 时间戳感知规则优于按影响力排序的替代方案。形式化地, 我们有:

$$\text{S-Tokens} = \text{Merge}(\tilde{\mathbf{S}}_1, \dots, \tilde{\mathbf{S}}_n) \in \mathbb{R}^{L_S \times d}, \quad L_S = \sum_{i=1}^n L_i + L_{\text{SEP}}. \quad (10)$$

3.3 OneTrans 块

如图 2(b) 所示, 每个 ONETRANS 块都是作用于归一化 token 序列的预归一化因果 Transformer: 先是 L_S 个序列 S-token, 随后是 L_{NS} 个非序列 NS-token。受异构特征组相关发现 [11] 的启发, 我们对 Transformer 进行轻量修改, 使其支持混合参数方案, 见图 2(c)。具体而言, 同构 S-token 共享一组参数。来自不同来源且语义各异的 NS-token 则使用 token 特定参数。

与 LLM 输入不同, 推荐系统中的 token 序列将序列 S-token 与多样化的 NS-token 组合在一起, 后者的取值范围和统计特性差异显著。由于这些差异, 后

归一化设置可能导致注意力坍塌和训练不稳定。为避免这一问题, 我们对所有 token 应用 RMSNorm [33] 作为预归一化, 以对齐不同 token 类型的尺度并稳定优化过程。

3.3.1 混合(共享/token 特定)因果注意力. ONETRANS

采用带因果注意力掩码的标准多头注意力(MHA); 唯一的变化在于 Q/K/V 的参数化方式。令 $\mathbf{x}_i \in \mathbb{R}^d$ 表示第 i 个 token。计算 Q/K/V 时, 我们对 S-token ($i \leq L_S$) 使用共享投影, 对 NS-token ($i > L_S$) 使用 L_{NS} 个 token 特定投影:

$$(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i) = (\mathbf{W}_i^Q \mathbf{x}_i, \mathbf{W}_i^K \mathbf{x}_i, \mathbf{W}_i^V \mathbf{x}_i), \quad (11)$$

其中, $\mathbf{W}_i^\Psi$ ($\Psi \in \{Q, K, V\}$) 采用混合参数化方案:

$$\mathbf{W}_i^\Psi = \begin{cases} \mathbf{W}_S^\Psi, & i \leq L_S \quad (\text{S-token 共享}), \\ \mathbf{W}_{NS,i}^\Psi, & i > L_S \quad (\text{NS-token 特定}). \end{cases} \quad (12)$$

注意力采用标准因果掩码, 并将 NS-token 置于 S-token 之后。这带来以下效果: (1) **S 侧**。每个 S-token 仅关注更早的 S 个位置。对于时间戳感知序列, 每个事件都以其历史为条件; 对于时间戳无关序列(按意图排序, 例如购买 $\rightarrow$ 加购 $\rightarrow$ 点击/曝光), 因果掩码使高意图信号能够影响并筛选之后的低意图行为。(2) **NS 侧**。每个 NS-token 都关注完整的 S 历史, 相当于对序列证据进行目标注意力聚合; 同时还关注此前的 NS-token, 从而提高 token 级交互的多样性。(3) **金字塔支持**。在 S 侧和 NS 侧, 因果掩码都会使信息逐渐向较后位置集中, 从而自然支持逐层剪枝 token 的金字塔调度, 详见下文。

3.3.2 混合(共享/token 特定) FFN. 类似地, 前馈网络采用相同的参数化策略: NS-token 使用 token 特定 FFN, S-token 使用共享 FFN,

$$\text{MixedFFN}(\mathbf{x}_i) = \mathbf{W}_i^2 \phi(\mathbf{W}_i^1 \mathbf{x}_i). \quad (13)$$

此处 $\mathbf{W}_i^1$ 和 $\mathbf{W}_i^2$ 遵循式(12)的混合参数化, 即 $i \leq L_S$ 共享参数, 而 $i > L_S$ 使用 token 特定参数。

综上, 相较标准因果 Transformer, ONETRANS 仅改变了参数化方式: NS-token 使用 token 特定的

QKV 和 FFN; S-token 则共享同一组参数。单一因果掩码将整个序列连接起来, 使 NS-token 能够聚合完整行为历史, 同时保留高效的 Transformer 风格计算。

3.4 金字塔堆栈

如第 3.3 节所述, 因果掩码会使信息集中到较后的位置。利用这种近因结构, 我们采用金字塔式调度: 在每个 ONETRANS 块层中, 只有最近的一部分 S-token 发出查询, 而键和值仍基于完整序列计算; 查询集合随深度增加而缩小。

令 $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^L$ 为输入 token 列表, 并令 $\mathcal{Q} = \{L-L'+1, \dots, L\}$ 表示满足 $L' \leq L$ 的尾部索引集合。遵循公式 12, 我们将查询修改为 $i \in \mathcal{Q}$:

$$\mathbf{q}_i = \mathbf{W}_i^Q \mathbf{x}_i, \quad i \in \mathcal{Q}, \quad (14)$$

而键和值仍按常规方式在完整序列 $\{1, \dots, L\}$ 上计算。注意力计算后, 仅保留 $i \in \mathcal{Q}$ 对应的输出, 将 token 长度缩减为 L' , 并在各层之间形成金字塔层次结构。

该设计带来两项优势: (i) 渐进式提炼: 较长的行为历史被汇聚到一小段尾部查询中, 使模型容量聚焦于信息量最大的事件, 并将信息整合到 NS-token 中; 以及 (ii) 计算效率: 注意力开销变为 $O(LL'd)$, FFN 则随 L' 线性扩展。缩小查询集合可直接减少 FLOPs 和激活内存。

3.5 训练与部署优化

3.5.1 跨请求 KV 缓存. 在工业 RecSys 中, 无论训练还是服务, 来自同一请求的样本都会连续处理: 不同候选的 S-token 保持相同, 而 NS-token 随候选物品变化。利用这一结构, 我们将广泛采用的 KV 缓存 [1] 集成到 ONETRANS 中, 形成统一的两阶段范式。

阶段 I (S 侧, 每个请求一次). 使用因果掩码处理所有 S-token, 并缓存其键值对和注意力输出。该阶段对每个请求只执行一次。

阶段 II (NS 侧, 每个候选一次)。对于每个候选, 计算其 NS-token, 并对缓存的 S 侧键和值执行交叉注意力, 随后经过 token 专用的 FFN 层。特别地, 候选特定序列 (例如 SIM [21]) 无法复用共享的 S 侧缓存, 因此会先通过池化聚合为 NS-token。

KV 缓存将 S 侧计算摊销到多个候选上, 使每个候选的计算保持轻量, 并消除冗余计算, 从而显著提升吞吐量。

由于用户行为序列仅追加, 我们将 KV 缓存扩展到跨请求场景: 每个新请求复用先前的缓存, 只为新加入的行为计算增量键和值。这使每个请求的序列计算量从 $O(L)$ 降至 $O(\Delta L)$, 其中 ΔL 是自上次请求以来新增行为的数量。

3.5.2 统一的 LLM 优化。 我们采用 FlashAttention-2 [8], 通过分块与核融合减少注意力 I/O 以及常规注意力的二次方激活占用, 从而在训练和推理中实现更低的内存使用量与更高的吞吐量。为进一步缓解内存压力, 我们将混合精度训练 (BF16/FP16) [18] 与激活重计算 [10] 结合使用; 后者会丢弃选定的前向激活, 并在反向传播期间重新计算。该组合以适度的额外计算换取显著的内存节省, 使模型无需改变架构便可采用更大的批量和更深的层数。

4 实验

通过离线评估和在线测试, 我们旨在回答以下研究问题 (RQ): **RQ1: 统一堆栈与先编码、后交互的对比**。在计算量相当条件下, 单一 Transformer 堆栈能否带来一致的性能提升? **RQ2: 哪些设计选择至关重要?** 我们对输入层 (例如分词器、序列融合) 和 ONETRANS 块 (例如参数共享、注意力类型、金字塔堆叠) 进行消融实验, 以评估不同设计选择对性能与效率的重要性。**RQ3: 系统效率**。在相同的 ONETRANS 计算图下, 金字塔堆叠、跨请求 KV 缓存、FlashAttention-2 以及结合重计算的混合精度能否降低 FLOPs、内存占用和延迟? **RQ4: 扩展定律**。当我们扩展长度 (token 序列长度)、宽度 (d_{model}) 和深度 (层数) 时, 损失/性能是否呈现预期的对数线性趋势? **RQ5: 在线 A/B**

表 1: OneTrans 实验所用数据集概览。

指标	数值
# 曝光量 (样本)	29.1B
# 用户数 (去重)	27.9M
# 物品数 (去重)	10.2M
每日曝光量 (均值 $\pm$ 标准差)	118.2M $\pm$ 14.3M
日活跃用户数 (均值 $\pm$ 标准差)	2.3M $\pm$ 0.3M

测试。在生产延迟约束下, 在线部署 ONETRANS 能否在关键业务指标 (例如人均订单数、人均 GMV) 上带来统计显著的提升?

4.1 实验设置

4.1.1 数据集。在离线评估中, 我们使用生产日志, 在严格遵守隐私规范的前提下 (所有个人身份信息均已匿名化并进行哈希处理), 于大规模工业排序场景中评估 ONETRANS。数据按时间顺序划分, 所有特征均在曝光时刻生成快照, 以防止时间泄漏并确保线上与离线一致。标签 (例如点击和订单) 在与生产设置一致的固定窗口内聚合。表 1 汇总了数据集统计信息。

4.1.2 任务与指标。我们评估式 (2) 中定义的两个二分类排序任务: CTR 和 CVR。性能采用 AUC 和 UAUC (按曝光加权的用户级 AUC) 衡量。

下一批次评估。数据按时间顺序处理。对于每个小批次, 我们先 (i) 在评估模式下记录预测结果, 再 (ii) 使用同一批次进行训练。AUC 和 UAUC 基于每天的预测结果按日计算, 最后在各天之间进行宏平均。

效率指标。我们报告参数量 (不含稀疏嵌入的模型参数) 和 TFLOPs (批大小为 2048 时以 TFLOPs 计的训练计算量)。

4.1.3 基线。我们使用相同特征和匹配的计算预算, 构建业界标准的模型组合作为基线。在先编码后交互范式下, 我们从广泛使用的生产基线 DCNv2+DIN [27, 35] 出发, 逐步增强特征交互模块: DCNv2 $\rightarrow$ Wukong [32] $\rightarrow$ HiFormer [11] $\rightarrow$ RankMixer [37]。在固定 RankMixer

表 2: 离线效果 (CTR/CVR) 与效率; AUC/UAUC 越高越好。* 表示按时间顺序部署到我们生产环境中的模型: $DCNv2 + DIN \rightarrow RankMixer + DIN \rightarrow RankMixer + Transformer \rightarrow OneTrans_S \rightarrow OneTrans_L$

类型	模型	CTR		CVR (下单)		效率	
		AUC $\uparrow$	UAUC $\uparrow$	AUC $\uparrow$	UAUC $\uparrow$	参数量 (M)	TFLOPs
(1) 基础模型	DCNv2 + DIN (基础) *	0.79623	0.71927	0.90361	0.71955	10	0.06
(2) 特征交互	Wukong + DIN	+0.08%	+0.11%	+0.14%	+0.11%	28	0.54
	HiFormer + DIN	+0.11%	+0.18%	+0.23%	-0.20%	108	1.35
	RankMixer + DIN*	+0.27%	+0.36%	+0.43%	+0.19%	107	1.31
(3) 序列建模	RankMixer + StackDIN	+0.40%	+0.37%	+0.63%	-1.28%	108	1.43
	RankMixer + LONGER	+0.49%	+0.59%	+0.47%	+0.44%	109	1.87
	RankMixer + Transformer*	+0.57%	+0.90%	+0.52%	+0.75%	109	2.51
(4) 统一框架	ONETRANS _S *	+1.13%	+1.77%	+0.90%	+1.66%	91	2.64
	ONETRANS _L (默认) *	+1.53%	+2.79%	+1.14%	+3.23%	330	8.62

后, 我们再改变序列建模模块: StackDIN $\rightarrow$ Transformer [4] $\rightarrow$ LONGER [1]。

4.1.4 超参数设置。我们报告两种设置: **OneTrans_S** 使用 6 个堆叠的 ONETRANS 块, 宽度为 $d=256$ 、注意力头数为 $H=4$, 目标参数量为 $\approx 100M$ 。**OneTrans_L** 扩展至 8 层, 宽度为 $d=384$ 。

输入通过统一分词器进行处理 (针对多行为序列采用时间戳感知融合; 针对非序列特征采用自动拆分), 并使用一种启发式金字塔调度, 在每一层将序列查询 token 的数量从 1190 线性缩减至 12 (ONETRANS_S) / 从 1500 缩减至 16 (ONETRANS_L)。具体而言, 我们跨层线性减少序列查询 token 的数量, 将每层的 token 数量舍入到最接近的 32 的倍数, 并将顶层设置为与非序列 token 的数量一致。

优化与基础设施。我们采用不带权重衰减的双优化器策略: 稀疏嵌入使用 Adagrad ($\beta_1=0.1$, $\beta_2=1.0$) 优化, 稠密参数使用 RMSProp (lr=0.005, alpha=0.99999, momentum=0) 优化。我们采用大规模 Transformer 训练中常用的稳定化技术, 包括 Pre-Norm [29] 和全局梯度范数裁剪 [23]。训练期间, 每个 GPU 的批大小设为 2048, 稠密层和稀疏层的梯度裁剪阈值分别为

90 和 120。在线推理时, 我们对每个 GPU 采用较小的批大小 100, 以平衡吞吐量和延迟。训练在 16 块 H100 GPU 上采用数据并行 all-reduce。

4.2 RQ1: 性能评估

我们以 DCNv2+DIN 为比较基准, 它是本场景扩展前的生产基线 (表 2)。在先编码、后交互范式下, 独立扩展任一组件均有收益: 升级特征交互模块 (DCNv2 $\rightarrow$ Wukong $\rightarrow$ HiFormer $\rightarrow$ RankMixer) 或序列建模模块 (StackDIN $\rightarrow$ Transformer $\rightarrow$ LONGER), 都能持续提升 CTR AUC/UAUC 和 CVR AUC。在我们的系统中, 这些指标超过 +0.1% 的提升即被视为有意义, 而超过 +0.3% 的提升通常对应于线上 A/B 测试中具有统计显著性的效果。不过, 由于每位用户的样本量较小且波动性更高, 我们会谨慎看待 CVR UAUC。

转向统一设计后, ONETRANS_S 相较基线提升了 +1.13%/+1.77% (CTR AUC/UAUC) 和 +0.90%/+1.66% (CVR AUC/UAUC)。在参数规模相当的情况下, 它还以相近的训练 FLOPs (2.64T 对 2.51T) 优于 RankMixer+Transformer, 证明了统一建模的优

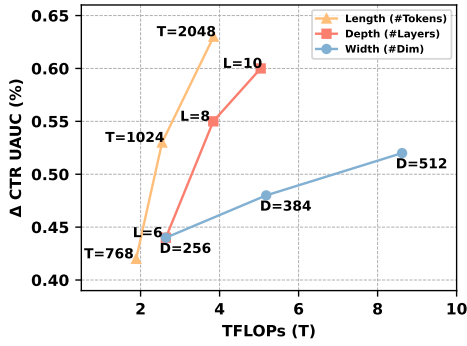
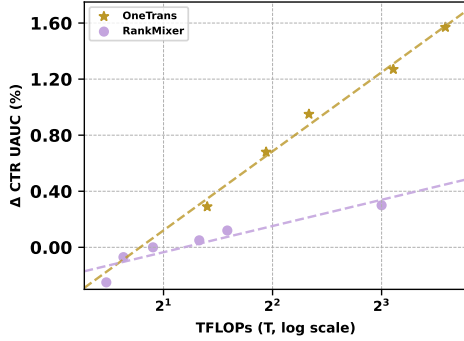
(a) 权衡: FLOPs 与 Δ UAUC(b) 缩放定律: Δ UAUC 与 FLOPs (对数) 的关系

图 3: 权衡关系与缩放定律的比较。

势。进一步扩展后, ONETRANS_L 取得了最佳总体提升: CTR AUC/UAUC 提升 +1.53%/+2.79%, CVR AUC/UAUC 提升 +1.14%/+3.23%, 表明随着模型容量增长, 质量表现具有可预测的提升趋势。

总之, 在单个 Transformer 中统一序列建模与特征交互, 比独立扩展任一组件都能带来更可靠且计算效率更高的改进。

4.3 RQ2: 通过消融研究分析设计选择

我们对所提出的 ONETRANS 模型进行消融研究, 以量化关键设计选择的贡献。完整结果汇总于表 3。我们评估以下变体: **输入变体**: i) 将 *Auto-Split Tokenizer* 替换为 *Group-wise Tokenizer* (第 1 行); ii) 使用时间戳无关的融合策略, 替代时间戳感知的序列融合 (第 2

行); iii) 在时间戳感知的序列融合中移除 [SEP] token (第 3 行); **OneTrans 块变体**: i) 在所有 token 之间共享同一组 Q/K/V 和 FFN 参数, 而不是为 NS-token 分配独立参数 (第 4 行); ii) 将因果注意力替换为全注意力 (第 5 行); iii) 在所有层均保留完整 token 序列, 从而禁用金字塔堆栈 (第 6 行)。

综上, 消融实验表明: 1) **Auto-Split Tokenizer** 相比手动将非序列特征分组为 token 具有明显优势, 这说明允许模型自动构建非序列 token 比依赖人工定义的特征分组更为有效; 2) 当时间戳存在时, **时间戳感知融合** 优于基于意图的排序, 这表明应优先考虑时间顺序而非事件影响; 3) 在时间戳无关融合下, 可学习的 [SEP] token 有助于模型区分不同序列; 4) NS-token 的 token 特定参数优于共享投影, 能够实现更好的特征辨别; 5) **因果注意力与全注意力** 表现相近, 但全注意力会使 KV 缓存等标准优化无法使用; 6) 在所有层保留全长 token 并无收益: ONETRANS 能够有效地将信息汇总到一个较短的尾部, 因此**金字塔设计**可以安全地剪除 query 以节省计算。此外, 在固定 TFLOPs 预算下, 相比全长设计, 金字塔设计支持接近 $1.75\times$ 倍的序列长度, 从而更充分地利用长度扩展带来的收益。

4.4 RQ3: 系统效率

为量化第 3.5 节所述优化, 我们以未经优化的 OneTrans_S 为基线进行消融, 并在表 4 中报告训练/推理指标。

如表所示: (i) **金字塔堆栈**通过剪除序列 query token, 同时降低训练成本 (运行时间/内存) 和服务开销 (p99 延迟/内存); (ii) **跨请求 KV 缓存**消除了冗余序列计算, 持续改善训练和服务中的运行时间/延迟及内存占用; (iii) **FlashAttention** 显著提升训练效率, 并小幅改善服务效率; (iv) **结合重计算的混合精度**带来最大的服务收益 (p99 延迟和推理内存), 同时也提高了训练效率。

这些结果证明了 LLM 优化在大规模推荐中的有效性。基于这些结果, 我们进一步扩展至 OneTrans_L , 并表明其在线效率仍与规模小得多的 DCNv2+DIN

表 3: 输入设计与 OneTrans 块设计选择的影响, 以 OneTrans_S 模型为参照。

类型	变体	CTR		CVR (订单)		效率	
		AUC↑	UAUC↑	AUC↑	UAUC↑	参数量 (M)	TFLOPs
输入	分组式分词器	-0.10%	-0.30%	-0.12%	-0.10%	78	2.35
	时间戳无关融合	-0.09%	-0.22%	-0.20%	-0.21%	91	2.64
	不含 Sep Token 的时间戳无关融合	-0.13%	-0.32%	-0.29%	-0.33%	91	2.62
OneTrans 块	共享参数	-0.15%	-0.29%	-0.14%	-0.29%	24	2.64
	全注意力	+0.00%	+0.01%	-0.03%	+0.06%	91	2.64
	不采用金字塔堆叠	-0.05%	+0.06%	-0.04%	-0.42%	92	8.08

表 4: 各变体相对于未经优化的 OneTrans_S 的影响。内存指 GPU 峰值使用量。

变体	训练		推理	
	运行时间 (ms)	内存 (GB)	延迟 (p99; ms)	内存 (GB)
未优化的 ONETRANS _S	407	53.13	54.00	1.70
+ 金字塔堆栈	-28.7%	-42.6%	-8.4%	-6.9%
+ 跨请求 KV 缓存	-30.2%	-58.4%	-29.6%	-52.9%
+ FlashAttention	-50.1%	-58.9%	-12.3%	-11.6%
+ 结合重计算的混合精度	-32.9%	-49.0%	-69.1%	-30.0%

表 5: OneTrans_L 与 DCNv2+DIN 基线之间的关键效率对比。

指标	DCNv2+DIN	OneTrans _L
TFLOPs	0.06	8.62
参数量 (M)	10	330
MFU	13.4	30.8
推理延迟 (p99, ms)	13.6	13.2
训练内存 (GB)	20	32
推理内存 (GB)	1.8	0.8

基线相当 (表 5), 凸显出统一的 Transformer 骨干网络能够直接采用 LLM 优化。

4.5 RQ4: 扩展定律验证

我们沿三个维度探究 ONETRANS 的缩放规律: (1) **长度**——输入 token 序列长度; (2) **深度**——堆叠块的数量; 以及 (3) **宽度**——隐藏状态维度。

如图 3(a) 所示, 增加长度可引入更多行为证据, 因此带来最大的提升。在深度与宽度之间, 我们观察到明显的权衡: 增加深度通常比单纯增大宽度带来更大的性能提升, 因为更深的堆栈能够提取更高阶的交互和更丰富的抽象表示。然而, 更深的模型也会增加串行计算, 而增大宽度更易于并行化。因此, 在深度和宽度之间做出选择时, 应在目标硬件预算下权衡性能收益与系统效率。

我们进一步通过同时增大 ONETRANS 的宽度和深度来分析缩放规律, 并将其与扩展 RankMixer+Transformer 基线作比较: 后者在 RankMixer 侧扩展至 1B。随后,

我们在对数尺度上绘制 ΔUAUC 与训练 FLOPs 的关系。如图 3(b) 所示, ONETRANS 和 RankMixer 均呈现明显的对数线性趋势, 但 ONETRANS 的斜率更陡。这可能是因为, 以 RankMixer 为中心的扩展缺少统一骨干, 并且其基于 MoE 的扩展主要只是增大 FFN 隐藏维度的宽度。综合来看, 这些结果表明 ONETRANS 具有更高的参数与计算效率, 能够为工业部署提供有利的性能—计算权衡。

尽管我们能够在严格的线上 p99 延迟约束下部署 ONETRANS_L, 但远超这一规模的扩展仍受线上效率限制; 进一步的系统—模型协同优化留待未来研究。

4.6 RQ5: 线上 A/B 测试

我们在两个大规模工业场景中评估 ONETRANS 的业务影响: (i) *Feeds* (首页信息流), 以及 (ii) *Mall* (包含 Feeds 和其他子场景的整体场景)。流量通过哈希和用户级随机化, 在用户/账户层面进行划分。对照模型与实验模型均使用过去 1.5 年的生产数据进行训练和部署, 以确保公平比较。

我们此前的生产基线 **RankMixer+Transformer** 作为对照组 (包含 $\approx 100\text{M}$ 个神经网络参数), 且不使用序列 KV 缓存。实验组部署 ONETRANS_L, 并采用第 3.5 节所述的服务优化。

我们报告用户级 **click/u**、**order/u** 和 **gmV/u** 相对于 *RankMixer+Transformer* 对照组的相对变化 ($\Delta\%$), 并给出双侧 95% 置信区间 (用户级分层 bootstrap); 同时报告**端到端延迟**, 其度量为从请求到达响应发出这一过程中, 单次曝光 p99 时间的相对变化 ($\Delta\%$; 越低越好)。如表 6 所示, ONETRANS_L 带来了稳定提升。在 *Feeds* 场景中, 它的 **click/u**、**order/u**、**gmV/u** 和延迟结果分别为 +7.737%、+4.3510%、+5.6848% 和 -3.91%。在 *Mall* 场景中, **click/u**、**order/u**、**gmV/u** 和延迟结果分别为 +5.143%、+2.5772%、+3.6696% 和 -3.26%。这些结果表明, 相较于强大的非统一基线, 统一建模框架在缩短服务时间的同时改善了业务指标。

表 6: 在线 A/B 测试结果: ONETRANS_L (实验组) 与 RankMixer+Transformer (对照组) 的比较。Click/u、Order/u 和 GMV/u 为相对变化 (%)。延迟为单次曝光端到端耗时的相对变化 $\Delta\%$ (越低越好)。* 表示 $p < 0.05$, ** 表示 $p < 0.01$

场景	click/u	order/u	gmV/u	延迟 (p99) ↓
Feeds	+7.737%**	+4.351%*	+5.685%*	-3.91%
Mall	+5.143%**	+2.577%**	+3.670%*	-3.26%

我们还观察到用户活跃天数增加了 +0.7478%, 并且冷启动商品 *order/u* 显著提升 +13.59%, 凸显了所提模型强大的泛化能力。

5 结论

我们提出 ONETRANS, 一种用于个性化排序的统一 Transformer 骨干网络, 以取代传统的编码-然后-交互范式。统一分词器将序列与非序列属性转换为一个 token 序列; 统一 Transformer 模块则通过为同质 (序列) token 共享参数、为异质 (非序列) token 使用 token 特定参数, 联合执行序列建模与特征交互。为使统一堆叠架构在大规模场景下保持高效, 我们采用逐步剪枝序列 token 的金字塔调度, 以及复用用户侧计算的跨请求 KV 缓存; 该设计还能进一步受益于 LLM 风格的系统优化 (例如 FlashAttention、混合精度)。在大规模评估中, 随着宽度/深度增加, ONETRANS 呈现近似对数线性的性能提升, 并在保持生产级延迟的同时带来具有统计显著性的业务提升。我们相信, 这种统一设计为扩展推荐系统提供了一条实用途径, 同时能够复用推动近期 LLM 进展的系统优化。

References

- [1] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, et al. 2025. LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders. *arXiv preprint arXiv:2505.04421* (2025).
- [2] Jianxin Chang, Chenbin Zhang, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. Pepnet: Parameter and embedding personalized network for infusing with personalized prior

- information. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3795–3804.
- [3] Yin-Wen Chang, Cho-Jui Hsieh, Kai-Wei Chang, Chih-Jen Lin, et al. 2010. Training and testing low-degree polynomial data mappings via linear svm. *Journal of Machine Learning Research* 11, 4 (2010).
 - [4] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior Sequence Transformer for E-commerce Recommendation in Alibaba. arXiv:1905.06874 [cs.IR] <https://arxiv.org/abs/1905.06874>
 - [5] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ipsir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. arXiv:1606.07792 [cs.LG] <https://arxiv.org/abs/1606.07792>
 - [6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
 - [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
 - [8] Tri Dao, Aleksander Thomas, Anima Anandkumar, Matei Zaharia, and Christopher Re. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691* (2023).
 - [9] Yufei Feng, Fuyu Lv, Weichen Shen, Menghan Wang, Fei Sun, Yu Zhu, and Keping Yang. 2019. Deep Session Interest Network for Click-Through Rate Prediction. arXiv:1905.06482 [cs.IR] <https://arxiv.org/abs/1905.06482>
 - [10] Andrunas Gruslys, Remi Munos, Ivo Daniel, Oriol Vinyals, and Koray Kavukcuoglu. 2016. Memory-Efficient Backpropagation through Time. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [11] Huan Gui, Ruoxi Wang, Ke Yin, Long Jin, Maciej Kula, Taibai Xu, Lichan Hong, and Ed H. Chi. 2023. Hiformer: Heterogeneous Feature Interactions Learning with Transformers for Recommender Systems. arXiv:2311.05884 [cs.IR] <https://arxiv.org/abs/2311.05884>
 - [12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, Xiuqiang He, and Zhenhua Dong. 2018. DeepFM: An End-to-End Wide & Deep Learning Framework for CTR Prediction. arXiv:1804.04950 [cs.IR] <https://arxiv.org/abs/1804.04950>
 - [13] Junjie Huang, Jizheng Chen, Jianghao Lin, Jiarui Qin, Ziming Feng, Weinan Zhang, and Yong Yu. 2024. A comprehensive survey on retrieval methods in recommender systems. *arXiv preprint arXiv:2407.21022* (2024).
 - [14] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781 [cs.IR] <https://arxiv.org/abs/1808.09781>
 - [15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
 - [16] Shichen Liu, Fei Xiao, Wenwu Ou, and Luo Si. 2017. Cascade ranking for operational e-commerce search. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1557–1565.
 - [17] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).
 - [18] Paulius Micikevicius, Sharan Narang, Jonah Alben, Greg Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *International Conference on Learning Representations (ICLR)*.
 - [19] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).
 - [20] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.
 - [21] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.
 - [22] Jiarui Qin, Jiachen Zhu, Bo Chen, Zhirong Liu, Weiwen Liu, Ruiming Tang, Rui Zhang, Yong Yu, and Weinan Zhang. 2022. Rankflow: Joint optimization of multi-stage cascade ranking systems as flows. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 814–824.
 - [23] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
 - [24] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. ACM, 1161–1170. doi:10.1145/3357384.3357925
 - [25] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. arXiv:1904.06690 [cs.IR] <https://arxiv.org/abs/1904.06690>
 - [26] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions.

- arXiv:1708.05123 [cs.LG] <https://arxiv.org/abs/1708.05123>
- [27] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference 2021 (WWW '21)*. ACM, 1785–1797. doi:10.1145/3442381.3450078
- [28] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.
- [29] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. 2020. On layer normalization in the transformer architecture. In *International conference on machine learning*. PMLR, 10524–10533.
- [30] Zhichen Zeng, Xiaolong Liu, Mengyue Hang, Xiaoyi Liu, Qinghai Zhou, Chaofei Yang, Yiqun Liu, Yichen Ruan, Laming Chen, Yuxin Chen, et al. 2024. Interformer: Towards effective heterogeneous interaction learning for click-through rate prediction. *arXiv preprint arXiv:2411.09852* (2024).
- [31] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [32] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. 2024. Wukong: Towards a scaling law for large-scale recommendation. *arXiv preprint arXiv:2403.02545* (2024).
- [33] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems* 32 (2019).
- [34] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2018. Deep Interest Evolution Network for Click-Through Rate Prediction. arXiv:1809.03672 [stat.ML] <https://arxiv.org/abs/1809.03672>
- [35] Guorui Zhou, Chengru Song, Xiaoqiang Zhu, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. arXiv:1706.06978 [stat.ML] <https://arxiv.org/abs/1706.06978>
- [36] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1079–1088.
- [37] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, Huizhi Yang, Zheng Chai, Zhe Chen, Yuchao Zheng, Qiwei Chen, Feng Zhang, Xun Zhou, Peng Xu, Xiao Yang, Di Wu, and Zuotao Liu. 2025. RankMixer: Scaling Up Ranking Models in Industrial Recommenders. arXiv:2507.15551 [cs.IR] <https://arxiv.org/abs/2507.15551>